

B.TECH II YEAR I SEM

BACKEND WEB TECHNOLOGIES

Question Bank - Complete Answers

How to use these notes

- ✓ **Short Questions = 2 Marks** - write the definition + 2 points (3 to 4 lines).
- ✓ **Long Questions = 5 Marks** - write: Definition/Introduction -> Points -> Diagram / Table -> Example / Code -> Conclusion.
- ✓ Always write the **heading in bold / underline** and draw the **diagram or table** - it fetches extra marks.
- ✓ For programs, write the code neatly and add 1-2 lines of **output / explanation**.
- ✓ Each module has 10 short + 10 long answers (Modules I to V = 50 short + 50 long).

Contents

Module	Topic
MODULE-I	Node.js, Express.js, Server & NPM
MODULE-II	SQLite, CRUD and RESTful APIs
MODULE-III	Authentication, JWT and Middleware
MODULE-IV	File System, Streams and Buffers
MODULE-V	MongoDB, Mongoose, MVC and JWT Authentication

MODULE-I : Node.js, Express.js, Server & NPM

PART A - SHORT QUESTIONS (2 Marks each)

Q1. What is meant by Backend development?

2 Marks

Backend development means building the **server-side** part of a web application – the part that users cannot see.

It handles business logic, database operations, authentication and APIs.

- Works behind the scenes: receives request -> processes -> talks to database -> sends response.
- Example: when you log in to a website, the backend checks your username and password in the database.

Q2. Explain briefly about Node.js.

2 Marks

Node.js is an **open-source, cross-platform JavaScript runtime environment** built on Google Chrome's **V8 engine**. It lets us run JavaScript outside the browser (on the server).

- **Event-driven and non-blocking I/O** – handles many requests at the same time.
- Uses a single thread with an event loop, so it is fast and lightweight.
- Comes with NPM (a huge library of packages).

Q3. What is the importance of Express.js?

2 Marks

Express.js is a **minimal and fast web framework for Node.js**. It makes building web servers and APIs easy.

- Provides easy **routing** (app.get, app.post) and **middleware** support.
- Reduces the code needed compared with the plain Node.js http module.
- Easy handling of request/response, JSON, static files, and errors.

Q4. What is the purpose of handling HTTP requests in web applications?

2 Marks

HTTP is the protocol used for communication between client (browser) and server. Handling HTTP requests means the server **receives the request, processes it and sends back a response**.

- It allows data exchange (GET, POST, PUT, DELETE) between client and server.
- It generates dynamic content, performs CRUD operations and sends proper status codes (200, 404, 500).

Q5. Explain briefly about Routing.

2 Marks

Routing means **deciding how the application responds to a client request for a particular URL (endpoint) and HTTP method**.

- Syntax: app.METHOD(path, handler)
- Example: app.get("/home", (req, res) => { res.send("Welcome"); });
- Different URLs (/home, /about, /users) can be mapped to different functions.

Q6. What is meant by Server?

2 Marks

A server is a **computer or a program that waits for client requests and sends back responses**.

- A web server stores website files / runs application code and serves them over HTTP.
- Examples of web servers: Apache, Nginx, Node.js HTTP server.
- Example: when you open google.com, Google's server sends the page to your browser.

Q7. What is Server Architecture?**2 Marks**

Server architecture is the **design or structure of how a server receives requests, processes them and sends responses**. It is based on the **client-server model**.

- It is usually divided into layers: Presentation layer (UI), Application layer (logic), Data layer (database).
- Common types: Monolithic, 3-tier and Microservices architecture.

Q8. What is Error Handling in Express.js? Why is it important?**2 Marks**

Error handling means **catching errors and sending a proper response instead of crashing the server**. In Express it is done with a special **error-handling middleware** that has four parameters: (err, req, res, next).

Importance:

- Prevents the application from crashing.
- Gives clear, user-friendly error messages and correct status codes.
- Helps in debugging and improves security (hides internal details).

Q9. What is NPM? Mention any two uses of NPM in Node.js.**2 Marks**

NPM (**Node Package Manager**) is the default package manager of Node.js. It is a huge online registry of reusable packages.

Two uses:

- **Install packages/libraries** – e.g. npm install express.
- **Manage dependencies and scripts** through the package.json file (npm start, npm run dev).

Q10. What is the difference between CommonJS and ES Modules in Node.js?**2 Marks**

Point	CommonJS (CJS)	ES Modules (ESM)
Import	const x = require('x')	import x from 'x'
Export	module.exports = ...	export / export default
Loading	Synchronous	Asynchronous
File type	.js (default), .cjs	.mjs or "type":"module"
Standard	Node.js specific	Official JavaScript standard

PART B - LONG QUESTIONS (5 Marks each)

Q1. Explain Backend Development. Discuss its role in web application architecture.

5 Marks

Definition

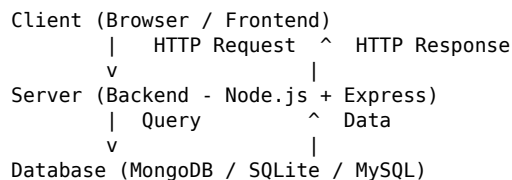
Backend development is the process of building the **server-side** of a web application. It contains the logic, database and APIs that run on the server and are **not visible to the user**. The frontend (what the user sees) talks to the backend to get or store data.

Main components of a backend

- **Server** – receives requests and sends responses (Node.js, Express).
- **Database** – stores data permanently (MySQL, SQLite, MongoDB).
- **APIs** – connect frontend and backend (REST API).
- **Business logic** – rules of the application (calculations, validations).
- **Authentication and Security** – login, tokens, permissions.

Role in web application architecture

A web application normally works in 3 tiers:



- **Receives requests** from the client and decides what to do.
- **Processes data** – applies business rules and validation.
- **Communicates with the database** – insert, read, update, delete.
- **Sends responses** (HTML/JSON) back to the client.
- **Provides security** – authentication, authorization, data protection.

Example

Online shopping site: when you click 'Place Order', the frontend sends a request. The backend checks the user, checks stock in the database, saves the order, and sends 'Order placed' back.

Popular backend technologies

Languages: Node.js (JavaScript), Python, Java, PHP. Frameworks: Express.js, Django, Spring Boot. Databases: MySQL, MongoDB, SQLite.

Q2. Explain CommonJS and ES Modules in detail.

5 Marks

A **module** is a separate file that contains code which can be reused in other files. Node.js supports two module systems.

1. CommonJS (CJS)

- Default module system of Node.js.
- Uses **require()** to import and **module.exports** to export.
- Loads modules **synchronously**.

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };
```

```
// app.js
const { add } = require('./math');
console.log(add(2, 3)); // 5
```

2. ES Modules (ESM)

- Official JavaScript (ES6) standard.
- Uses **import** and **export** keywords.
- Loads **asynchronously**; supports top-level await.
- To use it: save file as .mjs OR add "type": "module" in package.json.

```
// math.mjs
export function add(a, b) { return a + b; }
```

```
// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

Difference

Feature	CommonJS	ES Modules
Import	require()	import
Export	module.exports	export / export default
Loading	Synchronous	Asynchronous
Strict mode	Not by default	Always strict
__dirname	Available	Not available directly
Use	Older Node projects	Modern projects, browsers

Q3. Explain Server Architecture in detail and discuss its major features.

5 Marks

Definition

Server architecture is the **design and structure of a server system** – how it receives client requests, processes them using application logic and the database, and sends back responses. It follows the **client-server model**.

Layers of server architecture (3-tier)

- **Presentation layer** – client side (browser / mobile app); shows the UI.
- **Application (Logic) layer** – the server (Node.js/Express); contains business logic and APIs.
- **Data layer** – the database (MongoDB, MySQL, SQLite); stores data.

```
Client --request--> Web Server --> Application Logic --> Database
Client <--response-- Web Server <-- Application Logic <-- Database
```

Types of server architecture

- **Monolithic** – whole application in one codebase; simple but hard to scale.
- **Client-Server (2-tier / 3-tier)** – separate client, server and database.
- **Microservices** – application split into small independent services (user service, order service...).

Major features

- **Scalability** – can handle more users by adding resources.
- **Reliability / Availability** – server keeps working with minimum downtime.
- **Performance** – fast response using caching and non-blocking I/O.
- **Security** – authentication, authorization, HTTPS.
- **Maintainability** – code is organized in layers, so easy to modify.
- **Concurrency** – serves many clients at the same time.

Node.js server architecture

Node.js uses a **single-threaded event loop**. Incoming requests are placed in an event queue; the event loop picks them one by one, and long tasks (file/database) are done asynchronously, so the server never blocks.

Q4. Explain NPM Package Management in detail. Discuss its features and commands.

5 Marks

Introduction

NPM (Node Package Manager) is the default package manager for Node.js. It is used to install, update, share and manage packages (libraries) that our project needs. It has an online registry (npmjs.com) with millions of packages.

Features of NPM

- **Package installation** – download ready-made libraries easily.
- **Dependency management** – records all required packages in package.json.
- **Version control** – uses semantic versioning (e.g. 4.18.2).
- **Scripts** – run commands like npm start, npm test.
- **Global and local install** – packages for one project or for whole system.
- **Publishing** – developers can share their own packages.

Important files

- **package.json** – project details, scripts and dependency list.
- **package-lock.json** – exact versions of installed packages.
- **node_modules/** – folder where packages are stored.

Common commands

Command	Purpose
npm init -y	Create package.json with default values
npm install express	Install a package (local)
npm install	Install all dependencies of package.json
npm install -g nodemon	Install package globally
npm install --save-dev jest	Install as development dependency
npm update express	Update a package
npm uninstall express	Remove a package
npm list	Show installed packages
npm start / npm run dev	Run scripts from package.json

Example package.json

```
{
  "name": "myapp",
  "version": "1.0.0",
  "scripts": { "start": "node app.js" },
  "dependencies": { "express": "^4.18.2" }
}
```

Q5. Explain briefly about Request and Response Objects in Express.js.

5 Marks

Every route handler in Express receives two objects: **req (request)** and **res (response)**. Syntax: `app.get('/path', (req, res) => { ... })`

Request object (req)

Represents the HTTP request sent by the client. Important properties:

- **req.params** – route parameters (/users/:id -> req.params.id).
- **req.query** – query string values (/search?name=ravi -> req.query.name).
- **req.body** – data sent in POST/PUT body (needs express.json() middleware).
- **req.headers** – HTTP headers; **req.method**, **req.url** – method and URL.

Response object (res)

Used to send the reply back to the client. Important methods:

- **res.send()** – sends text/HTML.
- **res.json()** – sends JSON data.
- **res.status(code)** – sets status code (200, 404...).
- **res.redirect(url)** – redirects to another URL.
- **res.sendFile()** – sends a file; **res.end()** – ends response.

Example

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/users/:id', (req, res) => {
  res.json({ id: req.params.id, city: req.query.city });
});

app.post('/users', (req, res) => {
  res.status(201).json({ message: 'Created', data: req.body });
});
app.listen(3000);
```

Request: GET /users/5?city=Hyderabad -> Response: {"id":"5","city":"Hyderabad"}

Q6. Briefly explain about Error Handling in Express.js.

5 Marks

Meaning

Error handling in Express means **detecting errors during request processing and sending a proper response** instead of crashing the server. Express has built-in error handling and also allows custom error-handling middleware.

Error-handling middleware

It is a middleware function with **four parameters: (err, req, res, next)**. It must be written **after all routes**.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  throw new Error('Something went wrong!');
});

// 404 handler
app.use((req, res) => {
  res.status(404).send('Page not found');
});

// Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).json({ error: err.message });
});
app.listen(3000);
```

Key points

- Synchronous errors are caught automatically by Express.
- For asynchronous code use **try...catch** and pass the error with **next(err)**.
- Use proper status codes: 400 (bad request), 404 (not found), 500 (server error).

Why important

- Server does not crash.
- User gets clear messages.
- Helps debugging and hides sensitive information.

Q7.

Explain the steps involved in setting up an HTTP server using Node.js and write a suitable program to create an HTTP server.

5 Marks

Steps

- **Step 1:** Install Node.js and check with `node -v`.
- **Step 2:** Create a project folder and a file, e.g. `server.js`.
- **Step 3:** Import the built-in **http** module using `require('http')`.
- **Step 4:** Create the server using **http.createServer()** with a callback (`req, res`).
- **Step 5:** Inside the callback set the status code and headers using **res.writeHead()**.
- **Step 6:** Send the response using **res.end()**.
- **Step 7:** Make the server listen on a port using **server.listen(port)**.
- **Step 8:** Run with `node server.js` and open `http://localhost:3000` in the browser.

Program

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Welcome to Node.js Server</h1>');
  } else if (req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('This is About page');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 - Page Not Found');
  }
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

Explanation

- `createServer()` takes a callback which runs for **every request**.
- `req` holds request details (`url`, `method`); `res` is used to reply.
- `listen(3000)` starts the server on port 3000.

Output: `/` shows 'Welcome to Node.js Server'; `/about` shows 'This is About page'; other URLs show 404.

Q8.

Explain how HTTP requests are handled in Node.js. Discuss different HTTP methods with examples.

5 Marks

How requests are handled

- The client (browser) sends an HTTP request to the server.
- Node's `http` server receives it and triggers the callback (**req, res**).
- Server reads **req.method** and **req.url** to decide what to do.
- Server processes (reads body, accesses database) and sends the response using **res.writeHead()** and **res.end()**.
- Node uses the **event loop**, so many requests are handled without blocking.

HTTP methods

Method	Purpose	Example
GET	Read / fetch data	GET /users
POST	Create new data	POST /users
PUT	Update (replace) data	PUT /users/1
PATCH	Partially update data	PATCH /users/1
DELETE	Delete data	DELETE /users/1

Example program handling methods

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'application/json');

  if (req.method === 'GET' && req.url === '/users') {
    res.end(JSON.stringify([ { id: 1, name: 'Ravi' } ]));
  }

  else if (req.method === 'POST' && req.url === '/users') {
    let body = '';
    req.on('data', chunk => { body += chunk; });
    req.on('end', () => {
      res.statusCode = 201;
      res.end(JSON.stringify({ message: 'User created', data: body }));
    });
  }

  else if (req.method === 'PUT') {
    res.end(JSON.stringify({ message: 'User updated' }));
  }
  else if (req.method === 'DELETE') {
    res.end(JSON.stringify({ message: 'User deleted' }));
  }
  else {
    res.statusCode = 404;
    res.end(JSON.stringify({ error: 'Not found' }));
  }
});
server.listen(3000);
```

In POST, the body comes in pieces, so we collect it using the **'data'** event and use it in the **'end'** event.

Q9. Explain the concept of routing in Express.js with different routes.

5 Marks

Definition

Routing is the process of **mapping a URL (path) and an HTTP method to a handler function**. Syntax: **app.METHOD(path, handler)**.

Types of routes with example

```
const express = require('express');
const app = express();
app.use(express.json());

// 1. Basic routes (different methods)
app.get('/', (req, res) => res.send('Home Page'));
app.post('/users', (req, res) => res.send('User added'));
app.put('/users/:id', (req, res) => res.send('User updated'));
app.delete('/users/:id', (req, res) => res.send('User deleted'));

// 2. Route parameters
app.get('/users/:id', (req, res) => {
  res.send('User id = ' + req.params.id);
});

// 3. Query parameters (/search?name=ravi)
app.get('/search', (req, res) => {
  res.send('Searching for ' + req.query.name);
});

// 4. app.all() - any method
app.all('/secret', (req, res) => res.send('Any method'));

// 5. Chained routes with app.route()
app.route('/book')
  .get((req, res) => res.send('Get book'))
  .post((req, res) => res.send('Add book'));

// 6. Wildcard / 404 route
app.use((req, res) => res.status(404).send('Not Found'));

app.listen(3000);
```

Router (modular routing)

express.Router() is used to keep routes in separate files:

```
// routes/user.js
const router = require('express').Router();
router.get('/', (req, res) => res.send('All users'));
module.exports = router;

// app.js
app.use('/users', require('./routes/user'));
```

Advantages

- Organizes code clearly.
- Each URL has its own handler.
- Easy to build REST APIs.

Q10. Discuss Express.js in detail with its features, advantages and example program.

5 Marks

Introduction

Express.js is a **fast, minimal and flexible web application framework for Node.js**. It is used to build web applications and REST APIs easily. Install: npm install express.

Features

- **Routing** – simple URL and method based routing.
- **Middleware** – functions that process request/response (logging, auth, parsing).
- **Template engines** – supports EJS, Pug for dynamic HTML.
- **Static files** – express.static() serves HTML, CSS, images.
- **Error handling** – built-in and custom error middleware.
- **JSON support** – express.json() parses JSON body easily.
- **Large community and many third-party packages.**

Advantages

- Very fast development with less code.
- Lightweight and high performance.
- Easy to learn (uses JavaScript on both frontend and backend).
- Easy database integration (MongoDB, MySQL, SQLite).
- Flexible and scalable; good for REST APIs and microservices.

Example program

```
const express = require('express');
const app = express();

app.use(express.json()); // middleware

app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.get('/student/:name', (req, res) => {
  res.json({ student: req.params.name });
});

app.post('/student', (req, res) => {
  res.status(201).json({ message: 'Student added', data: req.body });
});

app.listen(3000, () => console.log('Server started on port 3000'));
```

Run: node app.js. Open http://localhost:3000/student/Ravi -> {"student":"Ravi"}.

MODULE-II : SQLite, CRUD and RESTful APIs

PART A - SHORT QUESTIONS (2 Marks each)

Q1. Define SQLite. Mention any two features of SQLite.

2 Marks

SQLite is a **lightweight, serverless, file-based relational database** that stores the whole database in a single file (e.g. student.db).

Features:

- **Serverless and zero-configuration** – no separate server to install.
- **Small, fast and portable** – whole database is one cross-platform file; supports standard SQL.

Q2. What is a Database Schema?

2 Marks

A database schema is the **blueprint or structure of a database**. It defines tables, columns, data types, primary/foreign keys and relationships.

- Example: student(id INTEGER PRIMARY KEY, name TEXT, age INTEGER)
- It tells how data is organized, but does not contain the data itself.

Q3. Expand CRUD and list its operations.

2 Marks

CRUD = **Create, Read, Update, Delete** – the four basic operations on data.

- **Create** – INSERT (POST)
- **Read** – SELECT (GET)
- **Update** – UPDATE (PUT/PATCH)
- **Delete** – DELETE (DELETE)

Q4. What is a RESTful API?

2 Marks

A RESTful API (REST = **Representational State Transfer**) is a web service that lets client and server communicate over HTTP using **URLs (resources)** and **HTTP methods** (GET, POST, PUT, DELETE).

- It is **stateless** and usually sends data in **JSON** format.
- Example: GET /students returns all students.

Q5. Differentiate between Query Parameters and Route Parameters.

2 Marks

Point	Route Parameter	Query Parameter
Position	Part of the URL path	After ? in the URL
Example	/users/10	/users?age=20&city=Hyd
Access	req.params.id	req.query.age
Use	Identify a specific resource	Filter, search, sort, paginate
Required	Usually mandatory	Usually optional

Q6. What is the purpose of a Primary Key in a database?

2 Marks

A primary key is a column (or set of columns) that **uniquely identifies each row** in a table.

- It cannot be NULL and cannot have duplicate values.
- Used to find, update and delete a particular record quickly and to link tables. Example: id in a student table.

Q7. What is request validation? Why is it important in API development?**2 Marks**

Request validation means **checking the data sent by the client** (body, params, query) for correctness before processing it.

Importance:

- Prevents wrong or empty data from entering the database.
- Improves **security** (stops SQL injection / malicious input) and avoids server errors; returns clear 400 error messages.

Q8. What is response formatting in REST APIs? Mention two commonly used response formats.**2 Marks**

Response formatting means **sending the API output in a standard, structured format** along with a proper status code so that clients can easily read it.

Two common formats:

- **JSON** (JavaScript Object Notation) – most popular.
- **XML** (Extensible Markup Language).

Q9. What is API testing? Name any two tools used for testing REST APIs.**2 Marks**

API testing is the process of **sending requests to API endpoints and checking whether the responses (data, status code, speed) are correct**, without using the user interface.

Tools: **Postman** and **Thunder Client** (VS Code). Others: Insomnia, cURL, Swagger.

Q10. Define query optimization?**2 Marks**

Query optimization is the process of **writing and tuning database queries so that they run faster and use fewer resources**.

- Techniques: use indexes, select only needed columns (avoid SELECT *), use WHERE and LIMIT, use proper joins.

PART B - LONG QUESTIONS (5 Marks each)

Q1. Explain SQLite integration in Node.js with suitable code examples.

5 Marks

Introduction

SQLite is a serverless, file-based relational database. To use it in Node.js we install two packages: **sqlite3** (the driver) and **sqlite** (a promise wrapper that lets us use async/await).

Steps

- 1. Create project: `npm init -y`
- 2. Install packages: `npm install express sqlite sqlite3`
- 3. Import modules and open the database using **open()**.
- 4. Create table (schema) with **db.run()**.
- 5. Start the server only after the connection succeeds.

Code

```
const express = require('express');
const { open } = require('sqlite');
const sqlite3 = require('sqlite3');
const path = require('path');

const app = express();
const dbPath = path.join(__dirname, 'student.db');
let db = null;

const initializeDBAndServer = async () => {
  try {
    db = await open({ filename: dbPath, driver: sqlite3.Database });

    await db.run(`CREATE TABLE IF NOT EXISTS student (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      age INTEGER
    )`);

    app.listen(3000, () => console.log('Server running on 3000'));
  } catch (e) {
    console.log('DB Error: ' + e.message);
    process.exit(1);
  }
};

initializeDBAndServer();
```

Important methods

Method	Use
<code>db.run(sql, params)</code>	INSERT, UPDATE, DELETE, CREATE (no rows returned)
<code>db.get(sql, params)</code>	Get a single row
<code>db.all(sql, params)</code>	Get all matching rows (array)
<code>db.exec(sql)</code>	Run multiple SQL statements

Note: Use placeholders (?) in queries to prevent SQL injection.

Q2. Describe CRUD operations in Node.js using SQLite with examples.

5 Marks

CRUD = Create, Read, Update, Delete. Assume the DB connection **db** is already open and a table **student(id, name, age)** exists.

1. Create (INSERT)

```
app.post('/students', async (req, res) => {
  const { name, age } = req.body;
  const result = await db.run(
    'INSERT INTO student(name, age) VALUES (?, ?)', [name, age]);
  res.status(201).send({ id: result.lastID, message: 'Student added' });
});
```

2. Read (SELECT)

```
// all students
app.get('/students', async (req, res) => {
  const rows = await db.all('SELECT * FROM student');
  res.send(rows);
});
// one student
app.get('/students/:id', async (req, res) => {
  const row = await db.get('SELECT * FROM student WHERE id = ?',
    [req.params.id]);
  res.send(row);
});
```

3. Update (UPDATE)

```
app.put('/students/:id', async (req, res) => {
  const { name, age } = req.body;
  await db.run('UPDATE student SET name = ?, age = ? WHERE id = ?',
    [name, age, req.params.id]);
  res.send('Student updated');
});
```

4. Delete (DELETE)

```
app.delete('/students/:id', async (req, res) => {
  await db.run('DELETE FROM student WHERE id = ?', [req.params.id]);
  res.send('Student deleted');
});
```

Mapping

CRUD	SQL	HTTP	db method
Create	INSERT	POST	db.run
Read	SELECT	GET	db.get / db.all
Update	UPDATE	PUT	db.run
Delete	DELETE	DELETE	db.run

Q3. Explain Query Writing and Query Optimization techniques in Node.js.

5 Marks

Query writing

Query writing means writing correct SQL statements in Node.js to work with the database. Always write them using **parameterized queries (?)**.

```
// SELECT with condition
const rows = await db.all(
  'SELECT name, age FROM student WHERE age > ? ORDER BY name', [18]);

// INSERT
await db.run('INSERT INTO student(name, age) VALUES (?, ?)', ['Ravi', 20]);

// JOIN
const data = await db.all(`SELECT s.name, c.course_name
  FROM student s JOIN course c ON s.course_id = c.id`);
```

- Use **WHERE** to filter, **ORDER BY** to sort, **LIMIT** to restrict rows, **JOIN** to combine tables.
- Use placeholders (?) instead of string concatenation to avoid **SQL injection**.

Query optimization

Query optimization means making queries **faster and efficient**. Techniques:

- **Use indexes** on columns used in WHERE / JOIN: CREATE INDEX idx_name ON student(name);
- **Select only required columns** – avoid SELECT *.
- **Use LIMIT and pagination** – do not fetch thousands of rows.
- **Use proper joins** instead of many separate queries.
- **Avoid queries inside loops** (N+1 problem) – use one query with IN or JOIN.
- **Use transactions** (BEGIN ... COMMIT) for many inserts.

- Use **EXPLAIN QUERY PLAN** to check how SQLite runs a query:

```
const plan = await db.all('EXPLAIN QUERY PLAN SELECT * FROM student WHERE name = ?', ['Ravi']);
console.log(plan);
```

Benefit

Faster response, less memory/CPU usage, better user experience and a scalable application.

Q4. Explain RESTful API Development in Node.js using Express.js with an example.

5 Marks

Introduction

A RESTful API exposes **resources** (like students, products) through URLs and uses **HTTP methods** to perform operations. Node.js + Express.js + a database (SQLite) is a popular way to build it. Data is exchanged as **JSON**.

Steps

- Setup project and install express, sqlite, sqlite3.
- Connect to database and create table.
- Use express.json() middleware to read JSON body.
- Create routes for each CRUD operation with correct HTTP methods.
- Send JSON responses with proper status codes.
- Test using Postman / Thunder Client.

Example - Student API

```
const express = require('express');
const { open } = require('sqlite');
const sqlite3 = require('sqlite3');
const app = express();
app.use(express.json());
let db;

(async () => {
  db = await open({ filename: 'student.db', driver: sqlite3.Database });
  await db.run(`CREATE TABLE IF NOT EXISTS student(
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, age INTEGER`);
  app.listen(3000);
})();

app.get('/students', async (req, res) => { // READ all
  res.json(await db.all('SELECT * FROM student'));
});
app.get('/students/:id', async (req, res) => { // READ one
  const s = await db.get('SELECT * FROM student WHERE id=?', [req.params.id]);
  s ? res.json(s) : res.status(404).json({ error: 'Not found' });
});
app.post('/students', async (req, res) => { // CREATE
  const { name, age } = req.body;
  const r = await db.run('INSERT INTO student(name,age) VALUES(?,?)', [name, age]);
  res.status(201).json({ id: r.lastID });
});
app.put('/students/:id', async (req, res) => { // UPDATE
  const { name, age } = req.body;
  await db.run('UPDATE student SET name=?, age=? WHERE id=?', [name, age, req.params.id]);
  res.json({ message: 'Updated' });
});
app.delete('/students/:id', async (req, res) => { // DELETE
  await db.run('DELETE FROM student WHERE id=?', [req.params.id]);
  res.json({ message: 'Deleted' });
});
```

Endpoints summary

Method	URL	Action	Status
GET	/students	List all	200
GET	/students/:id	Get one	200 / 404

Method	URL	Action	Status
POST	/students	Create	201
PUT	/students/:id	Update	200
DELETE	/students/:id	Delete	200

Q5. Discuss Query Handling in Express.js using query parameters.

5 Marks

Meaning

Query parameters are **key=value pairs added to the URL after '?'** and separated by '&'. Example: /students?age=20&city=Hyderabad. In Express they are read using **req.query** (an object). They are mainly used for **filtering, searching, sorting and pagination**.

Syntax

```
app.get('/students', (req, res) => {
  const { age, city } = req.query; // age = '20', city = 'Hyderabad'
  res.send(`Age: ${age}, City: ${city}`);
});
```

Example with SQLite

```
app.get('/students', async (req, res) => {
  const { name = '', minAge = 0 } = req.query;
  const rows = await db.all(
    `SELECT * FROM student WHERE name LIKE ? AND age >= ?`,
    [`%${name}%`, minAge]);
  res.json(rows);
});
// Request: GET /students?name=ra&minAge=18
```

Points to remember

- All query values come as **strings** – convert with Number() when needed.
- Query parameters are **optional**, so give default values.
- Always validate query values before using them in SQL.
- Multiple parameters are joined using &.

Q6. Explain the REST principles in detail with suitable examples.

5 Marks

REST (Representational State Transfer) is an architectural style for designing web APIs. It has the following principles (constraints):

1. Client-Server separation

Client (UI) and server (data/logic) are independent and communicate only through requests/responses. Example: the same API serves a website and a mobile app.

2. Stateless

Each request contains **all information** needed; the server does not remember previous requests. Example: token sent with every request.

3. Uniform Interface

Resources are identified by URLs and handled with standard HTTP methods. Example: GET /students, POST /students, DELETE /students/1 (use nouns, not verbs - not /getStudents).

4. Resource-based and representation

Everything is a resource; the server sends its representation (usually JSON): {"id":1,"name":"Ravi"}.

5. Cacheable

Responses can be marked cacheable to reduce server load and improve speed. Example: Cache-Control header.

6. Layered System

Between client and server there can be layers (load balancer, proxy, security) – client does not know about them.

7. Code on Demand (optional)

Server may send executable code (e.g. JavaScript) to the client.

Good REST practices

Operation	HTTP Method	URL	Status
Get all	GET	/students	200
Create	POST	/students	201
Update	PUT	/students/5	200
Delete	DELETE	/students/5	200 / 204

Q7. Explain how to implement searching, filtering, sorting and pagination in a RESTful API using Node.js, Express.js and SQLite.

5 Marks

All four features are implemented with **query parameters** (req.query) and a dynamic SQL query. Table: product(id, name, category, price).

Meaning

- **Searching** – find records matching a word: ?search=phone (SQL: LIKE).
- **Filtering** – restrict by a field value: ?category=mobile (SQL: WHERE).
- **Sorting** – order the results: ?sortBy=price&order=desc (SQL: ORDER BY).
- **Pagination** – return data page by page: ?page=2&limit=5 (SQL: LIMIT, OFFSET).

Code

```
app.get('/products', async (req, res) => {
  const {
    search = '', category = '',
    sortBy = 'id', order = 'ASC',
    page = 1, limit = 5
  } = req.query;

  // allow only safe column names for sorting
  const allowed = ['id', 'name', 'price'];
  const sortCol = allowed.includes(sortBy) ? sortBy : 'id';
  const sortOrder = order.toUpperCase() === 'DESC' ? 'DESC' : 'ASC';

  const offset = (Number(page) - 1) * Number(limit);

  const sql = `SELECT * FROM product
    WHERE name LIKE ? AND category LIKE ?
    ORDER BY ${sortCol} ${sortOrder}
    LIMIT ? OFFSET ?`;

  const rows = await db.all(sql,
    [`%${search}%`, `%${category}%`, Number(limit), offset]);
  res.json({ page: Number(page), count: rows.length, data: rows });
});
```

Sample request

GET /products?search=pho&category=mobile&sortBy=price&order=desc&page=1&limit=5

This returns the first 5 mobile products whose name contains 'pho', sorted by price (high to low).

Formula

OFFSET = (page - 1) x limit. Column names cannot be passed as ? placeholders, so they are checked against an allowed list to avoid SQL injection.

Q8.**Explain request validation in REST APIs. Demonstrate how to validate user input before inserting data into a SQLite database.****5 Marks**

Meaning

Request validation is **checking the incoming data (body, params, query)** to make sure it is present, of correct type and in valid format **before** using it. If invalid, the server returns **400 Bad Request**.

Why needed

- Keeps the database clean (no empty/wrong values).
- Prevents attacks like SQL injection.
- Avoids server crashes.
- Gives helpful messages to the client.

Common checks

- Required fields present.
- Correct data type (age is a number).
- Length / range (name min 3 chars, age 1-100).
- Format (valid email).
- Duplicate check (unique email).

Example

```
app.post('/students', async (req, res) => {
  const { name, age, email } = req.body;
  const errors = [];

  if (!name || name.trim().length < 3)
    errors.push('Name must have at least 3 characters');
  if (!Number.isInteger(age) || age < 1 || age > 100)
    errors.push('Age must be a number between 1 and 100');
  if (!email || !/^[@\s]+\.[^\s]+\.[^\s]+$/.test(email))
    errors.push('Email is not valid');

  if (errors.length > 0) {
    return res.status(400).json({ errors });
  }

  const exist = await db.get('SELECT id FROM student WHERE email = ?', [email]);
  if (exist) return res.status(409).json({ error: 'Email already exists' });

  const r = await db.run(
    'INSERT INTO student(name, age, email) VALUES (?, ?, ?)',
    [name.trim(), age, email]);
  res.status(201).json({ id: r.lastID, message: 'Student added' });
});
```

Libraries such as **express-validator**, **Joi** or **Zod** can also be used. Also use DB constraints (NOT NULL, UNIQUE) as a second safety layer.

Q9.**Explain common debugging techniques for Node.js, Express.js and SQLite applications.****5 Marks**

Debugging means **finding and fixing errors (bugs)** in the code. Common techniques:

1. console.log()

Print variables, req.body, req.params, SQL results to see what is happening.

```
console.log('Body:', req.body);
console.log('Rows:', rows);
```

2. try...catch and error messages

```
try {
  await db.run('INSERT INTO student(name) VALUES (?)', [name]);
} catch (e) {
  console.log('DB Error:', e.message);
  res.status(500).send('Server error');
}
```

3. Node.js Debugger / VS Code debugger

Run **node --inspect app.js** or use VS Code breakpoints; step through code line by line and watch variables.

4. nodemon

Automatically restarts the server on file changes, so bugs are checked quickly (npm install -g nodemon).

5. Testing with Postman / Thunder Client

Send requests and check status code, headers and response body of each API.

6. Logging middleware (morgan)

app.use(morgan('dev')) prints every request method, URL, status and time.

7. Debugging SQLite

- Print the SQL query and parameters before running.
- Use **DB Browser for SQLite** or sqlite3 CLI to inspect tables and data.
- Use **EXPLAIN QUERY PLAN** to check slow queries.
- Check for common errors: wrong table/column name, missing file path, SQLITE_CONSTRAINT (duplicate/NULL).

8. Read the error / stack trace

The stack trace shows the file name and line number where the error happened; fix from the top-most line of your own code.

9. Global error-handling middleware

Use app.use((err, req, res, next) => {...}) to catch all errors in one place and log them.

Q10. Explain Query Parameters and Route Parameters in Express.js. Develop suitable API examples demonstrating both types of parameters.

5 Marks

Route parameters

Named parts of the URL path written with a colon (:). They identify a **specific resource**. Accessed using **req.params**. Example URL: /students/5 (route: /students/:id).

Query parameters

Key-value pairs after ? in the URL used for **filtering, searching, sorting**. Accessed using **req.query**. Example URL: /students?age=20&city=Hyd.

Difference

Point	Route Parameter	Query Parameter
Syntax	/students/:id	/students?age=20
Access	req.params	req.query
Purpose	Identify one resource	Filter / search / sort
Optional?	No (required)	Yes

Example API

```
const express = require('express');
const app = express();

const students = [
  { id: 1, name: 'Ravi', age: 20, city: 'Hyderabad' },
  { id: 2, name: 'Sita', age: 21, city: 'Chennai' },
  { id: 3, name: 'Anil', age: 20, city: 'Hyderabad' }
];

// ROUTE PARAMETER: GET /students/2
app.get('/students/:id', (req, res) => {
  const s = students.find(x => x.id === Number(req.params.id));
  s ? res.json(s) : res.status(404).json({ error: 'Student not found' });
});
```

```
// QUERY PARAMETERS: GET /students?age=20&city=Hyderabad
app.get('/students', (req, res) => {
  let result = students;
  if (req.query.age)
    result = result.filter(x => x.age === Number(req.query.age));
  if (req.query.city)
    result = result.filter(x => x.city === req.query.city);
  res.json(result);
});

// BOTH TOGETHER: GET /students/1/marks?sem=2
app.get('/:id/marks', (req, res) => {
  res.json({ studentId: req.params.id, semester: req.query.sem });
});
app.listen(3000);
```

Outputs

- /students/2 -> {"id":2,"name":"Sita",...}
 - /students?age=20&city=Hyderabad -> Ravi and Anil
 - /students/1/marks?sem=2 -> {"studentId":"1","semester":"2"}
-

MODULE-III : Authentication, JWT and Middleware

PART A - SHORT QUESTIONS (2 Marks each)

Q1. What is JWT (JSON Web Token) authentication?

2 Marks

JWT is a **compact, digitally signed token** used to verify the identity of a user. After login the server creates a token and gives it to the client; the client sends it with every request (Authorization: Bearer <token>).

- It is **stateless** – the server need not store sessions.
- Example library: jsonwebtoken.

Q2. What are the three parts of a JWT?

2 Marks

A JWT has three parts separated by dots: **xxxxx.yyyyy.zzzzz**

- **Header** – token type (JWT) and algorithm (HS256).
- **Payload** – claims/data like user id, role, expiry (exp).
- **Signature** – created using header + payload + secret key to verify the token is not changed.

Q3. What is the purpose of the JWT signature?

2 Marks

The signature **ensures integrity and authenticity** of the token.

- It is created as HMACSHA256(header + "." + payload, secret).
- If anyone changes the header or payload, the signature will not match and the server rejects the token.

Q4. Define authentication and authorization.

2 Marks

- **Authentication** – process of **verifying who the user is** (login with username and password).
- **Authorization** – process of **deciding what the authenticated user is allowed to do** (access to admin page, delete rights).

Q5. List any two authentication strategies used in web applications.

2 Marks

- **Session-based (cookie) authentication** – server stores session, client keeps session id in a cookie.
- **Token-based (JWT) authentication** – client keeps a signed token and sends it in every request.

(Others: OAuth 2.0 / social login, API keys, Basic auth.)

Q6. What is middleware?

2 Marks

Middleware is a **function that runs between the request and the response** in Express. It has access to **req, res and next()**.

- It can modify req/res, end the request, or pass control to the next function using next().
- Example: app.use(express.json());

Q7. Mention any two uses of middleware functions in web applications.

2 Marks

- **Authentication / authorization** – check the JWT token before allowing access to routes.
- **Logging and parsing** – log each request, parse JSON body (express.json()), handle errors, enable CORS.

Q8. What is custom middleware?

2 Marks

Custom middleware is a **middleware function written by the developer** to perform a specific task.

Example:

- `const logger = (req, res, next) => { console.log(req.method, req.url); next(); }; app.use(logger);`
- Used for custom authentication, validation, logging and role checking.

Q9. What is error logging, and why is it important?**2 Marks**

Error logging means **recording errors (message, time, URL, stack trace) in the console or a log file** when they occur in the application.

Importance:

- Helps developers find and fix bugs quickly.
- Helps in monitoring, security auditing and analysing the application's behaviour in production.

Q10. Differentiate between authentication and authorization with one example each.**2 Marks**

Point	Authentication	Authorization
Question	Who are you?	What can you do?
Purpose	Verify identity	Give permissions / access
Order	Happens first	Happens after authentication
Based on	Credentials (password, token)	Roles / permissions
Example	Logging in with email + password	Only admin can delete users

PART B - LONG QUESTIONS (5 Marks each)

Q1. Explain JWT (JSON Web Token) authentication in detail. Describe the structure of a JWT, token generation, verification, expiration, and its advantages and limitations.

5 Marks

Introduction

JWT is an open standard (RFC 7519) for **securely transmitting information as a digitally signed JSON object**. It is widely used for stateless authentication in REST APIs. Install: `npm install jsonwebtoken`.

Structure of JWT

xxxxx.yyyyy.zzzzz = Header . Payload . Signature

- **Header** – {"alg":"HS256","typ":"JWT"} – algorithm and type.
- **Payload** – {"id":101,"role":"admin","exp":1700000000} – user data (claims).
- **Signature** – HMACSHA256(base64(header) + '.' + base64(payload), secret) – prevents tampering.
- Header and payload are only Base64 encoded (NOT encrypted), so never store passwords in them.

Token generation (jwt.sign)

```
const jwt = require('jsonwebtoken');
const token = jwt.sign(
  { id: user.id, role: user.role }, // payload
  process.env.JWT_SECRET,          // secret key
  { expiresIn: '1h' }               // expiration
);
```

Token verification (jwt.verify)

```
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  console.log(decoded.id, decoded.role);
} catch (err) {
  console.log('Invalid or expired token');
}
```

Expiration

The **exp** claim (set using `expiresIn`) decides how long the token is valid. After it expires, `jwt.verify` throws **TokenExpiredError** and the user must log in again (or use a refresh token). Short expiry improves security.

Working flow

1. Client sends username + password --> Server
2. Server verifies and creates JWT --> Client
3. Client stores token, sends it in header: Authorization: Bearer <token>
4. Server verifies token and sends data

Advantages

- **Stateless** – no session storage on server; scalable.
- Compact and fast; easy to send in headers.
- Works across different domains / mobile apps.
- Can carry user info (id, role) inside the token.

Limitations

- Token cannot be easily revoked before expiry.
- If stolen, anyone can use it until it expires.
- Payload is readable (only encoded).
- Larger than a session id; secret key must be kept safe.

Q2.

Discuss various authentication strategies used in web applications. Compare session-based authentication and JWT authentication.

5 Marks

Authentication strategies

- **1. Session-based (cookie) authentication** – after login the server creates a session, stores it in memory/database and sends a session id in a cookie.
- **2. Token-based (JWT) authentication** – server issues a signed token; client sends it with each request; no server storage.
- **3. OAuth 2.0 / Social login** – login using Google, Facebook, GitHub; a third party verifies the user.
- **4. API Key authentication** – a unique key identifies the application/client calling the API.
- **5. Basic authentication** – username and password sent (Base64) in the header; simple but less secure.
- **6. Multi-Factor Authentication (MFA)** – password + OTP/biometric for higher security.

Session-based flow

Login -> server creates session {id: abc123, user: Ravi} (stored on server)
 -> sends cookie: sessionId=abc123
 Next request -> browser sends cookie -> server looks up session -> allowed

JWT flow

Login -> server creates signed token (contains user id, role)
 -> client stores token
 Next request -> Authorization: Bearer <token> -> server verifies signature -> allowed

Comparison

Point	Session-based	JWT
Storage	Session stored on server	Token stored on client
State	Stateful	Stateless
Scalability	Harder (shared session store)	Easy to scale
Sent via	Cookie	Authorization header
Logout / revoke	Easy (delete session)	Hard (until expiry)
Mobile / API use	Less suitable	Very suitable
Memory use	Server memory needed	No server memory

Q3.

Explain the complete user authentication process in a web application, from user registration and password storage to login.

5 Marks

Overview

The process has two phases: **Registration (sign-up)** and **Login (sign-in)**. Passwords must never be stored as plain text; they are **hashed** using bcrypt.

A. Registration

- **1.** User fills name, email, password and submits (POST /register).
- **2.** Server **validates** the input (required fields, email format, password length).
- **3.** Server checks that the email is **not already registered**.
- **4.** Password is **hashed with bcrypt** (with salt): `bcrypt.hash(password, 10)`.
- **5.** User details with the **hashed password** are saved in the database.
- **6.** Server returns 201 Created.

```
app.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  const exist = await User.findOne({ email });
  if (exist) return res.status(409).json({ msg: 'User already exists' });

  const hashed = await bcrypt.hash(password, 10);
  await User.create({ name, email, password: hashed });
  res.status(201).json({ msg: 'Registered successfully' });
});
```


B. Login

- **1.** User sends email and password (POST /login).
- **2.** Server finds the user by email; if not found -> 401/404.
- **3.** Server compares entered password with stored hash: `bcrypt.compare()`.
- **4.** If wrong -> 401 Unauthorized.
- **5.** If correct -> server generates a **JWT** (or session) and sends it.
- **6.** Client stores the token and sends it for later requests.

```
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(401).json({ msg: 'Invalid credentials' });

  const ok = await bcrypt.compare(password, user.password);
  if (!ok) return res.status(401).json({ msg: 'Invalid credentials' });

  const token = jwt.sign({ id: user._id, role: user.role },
    process.env.JWT_SECRET, { expiresIn: '1h' });
  res.json({ token });
});
```

Password storage rules

- Never store plain text; store **hash + salt**.
- Hashing is one-way – original password cannot be recovered.
- Use `bcrypt` / `argon2`, not MD5 or SHA1.

Q4. What are middleware functions? Explain their purpose, working mechanism, execution flow, and importance in web application development with suitable examples.

5 Marks

Definition

Middleware functions are functions that have access to **req, res and next** and are executed **during the request-response cycle**, between receiving the request and sending the response.

```
function middleware(req, res, next) {
  // do something
  next();    // pass control to next middleware / route
}
```

Purpose

- Execute any code (e.g. logging).
- Modify req and res objects.
- End the request-response cycle (`res.send()`).
- Call the next middleware using `next()`.
- Authentication, validation, parsing, error handling.

Working mechanism and execution flow

Middleware runs **in the order in which it is written**. Each one either ends the response or calls **next()**.

```
Request --> [Logger] --next()--> [express.json()] --next()--> [Auth]
                                     |
Response <-- [Route Handler] <----- next() -----+
(If any middleware does not call next() and does not send a response, the request hangs.)
```

Types of middleware

- **Application-level** – `app.use(fn)`
- **Router-level** – `router.use(fn)`
- **Built-in** – `express.json()`, `express.static()`
- **Third-party** – `cors`, `morgan`, `helmet`
- **Error-handling** – (`err, req, res, next`)

Example

```
const express = require('express');
const app = express();

app.use(express.json()); // built-in

app.use((req, res, next) => { // custom logger
  console.log(req.method + ' ' + req.url);
  next();
});

app.get('/', (req, res) => res.send('Home'));

app.use((err, req, res, next) => { // error middleware
  res.status(500).send('Server Error');
});
app.listen(3000);
```

Importance

- Avoids repeating code in every route (reusability).
- Keeps code clean and modular.
- Adds security and validation easily.
- Helps in logging, debugging and error handling.

Q5. Explain the implementation of authentication middleware for protecting routes. Describe how middleware validates JWT tokens and allows or denies access to protected resources.

5 Marks

Concept

An authentication middleware **checks the JWT token of every request before it reaches a protected route**. If the token is valid -> access is allowed; otherwise -> access is denied (401/403).

Steps of validation

- **1.** Read the token from header: Authorization: Bearer <token>.
- **2.** If token is missing -> respond **401 Unauthorized**.
- **3.** Verify token using `jwt.verify(token, secret)`.
- **4.** If invalid/expired -> respond **403 Forbidden** (or 401).
- **5.** If valid -> attach decoded user to `req.user` and call **next()**.

Middleware code

```
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  const authHeader = req.headers['authorization']; // 'Bearer <token>'
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ message: 'Access denied. No token' });
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // available in next handlers
    next();
  } catch (err) {
    return res.status(403).json({ message: 'Invalid or expired token' });
  }
};
module.exports = authMiddleware;
```

Protecting routes

```
const auth = require('./authMiddleware');

app.get('/public', (req, res) => res.send('Anyone can see'));

app.get('/profile', auth, (req, res) => { // protected
  res.json({ message: 'Welcome', user: req.user });
});
```

```
});
```

Result

Request	Result
No token	401 Unauthorized
Wrong / expired token	403 Forbidden
Valid token	200 OK - protected data returned

Q6. What is Custom Middleware Development? Explain how to create and implement custom middleware for authentication, request validation, logging, and access control.

5 Marks

Meaning

Custom middleware is a **user-defined function (req, res, next)** created to handle a specific task in the request cycle. We write it once and reuse it using `app.use()` (all routes) or in a single route.

1. Logging middleware

```
const logger = (req, res, next) => {  
  console.log(`${new Date().toISOString()} ${req.method} ${req.url}`);  
  next();  
};  
app.use(logger);
```

2. Request validation middleware

```
const validateUser = (req, res, next) => {  
  const { name, email } = req.body;  
  if (!name || !email) {  
    return res.status(400).json({ error: 'Name and email are required' });  
  }  
  next();  
};  
app.post('/users', validateUser, (req, res) => res.send('User created'));
```

3. Authentication middleware

```
const authenticate = (req, res, next) => {  
  const token = req.headers.authorization?.split(' ')[1];  
  if (!token) return res.status(401).json({ error: 'Login required' });  
  try {  
    req.user = jwt.verify(token, process.env.JWT_SECRET);  
    next();  
  } catch { res.status(403).json({ error: 'Invalid token' }); }  
};
```

4. Access control (role-based) middleware

```
const authorize = (...roles) => (req, res, next) => {  
  if (!roles.includes(req.user.role)) {  
    return res.status(403).json({ error: 'Access denied' });  
  }  
  next();  
};  
  
app.delete('/users/:id', authenticate, authorize('admin'),  
  (req, res) => res.send('User deleted'));
```

Rules to remember

- Always call `next()` or send a response.
- Order matters – put `authenticate` before `authorize`.
- Use `app.use()` for global, or pass in the route for specific middleware.

Q7.

Explain Error Logging in web applications. Discuss different types of errors, logging levels, error-handling middleware.**5 Marks**

Error logging

Error logging is **recording information about errors (message, time, URL, user, stack trace)** to the console or log files/services, so developers can analyse and fix problems.

Types of errors

- **Syntax errors** – mistakes in code (missing bracket).
- **Runtime errors** – happen while running (undefined variable, null).
- **Logical errors** – program runs but gives wrong output.
- **Validation errors** – invalid user input (400).
- **Authentication / Authorization errors** – 401 / 403.
- **Database errors** – connection failed, duplicate key.
- **Server errors** – unexpected failure (500).

Logging levels

Level	Meaning
error	Serious failure that needs attention
warn	Something unexpected, but app continues
info	Normal events (server started, user login)
debug	Detailed information for developers

Error-handling middleware with logging

```
const fs = require('fs');

// 404 handler
app.use((req, res, next) => {
  const err = new Error('Route not found');
  err.status = 404;
  next(err);
});

// Error handler (4 parameters)
app.use((err, req, res, next) => {
  const log = `${new Date().toISOString()} [ERROR] ${req.method} ${req.url}`
    + ` - ${err.message}\n`;
  console.error(log);
  fs.appendFile('error.log', log, () => {}); // save in file

  res.status(err.status || 500).json({
    success: false,
    message: err.message
  });
});
```

Tools

For production use libraries like **winston** or **morgan** for structured logging with levels and files.

Importance

- Easy debugging and quick bug fixing.
- Monitoring application health.
- Security auditing (detect failed logins).
- Never send stack traces to users in production.

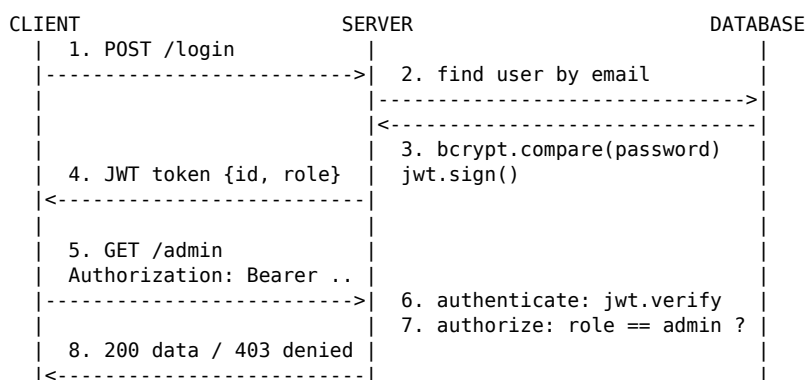
Q8. Design an authentication and authorization architecture for a web application using JWT, middleware, user management, and role-based authorization. Explain the flow with a suitable diagram.

5 Marks

Components of the architecture

- **Client** – browser / mobile app; stores the JWT.
- **Auth routes** – /register and /login (public).
- **User management** – User model {name, email, password(hash), role}.
- **JWT service** – creates and verifies tokens.
- **Authentication middleware** – verifies token, sets req.user.
- **Authorization middleware (RBAC)** – checks the role of the user.
- **Database** – stores users and their roles (user, admin).
- **Error handler** – returns 401 / 403 / 500.

Flow diagram



Implementation (short)

```
// authenticate: verify token -> req.user = { id, role }
// authorize: allow only given roles
const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next()
  : res.status(403).json({ message: 'Forbidden' });

app.get('/profile', authenticate, (req, res) => res.json(req.user));
app.get('/admin/users', authenticate, authorize('admin'), getAllUsers);
app.delete('/users/:id', authenticate, authorize('admin'), deleteUser);
```

Role-Based Access Control (RBAC)

Route	user	admin
/profile	Allowed	Allowed
/admin/users	Denied (403)	Allowed
DELETE /users/:id	Denied (403)	Allowed

Q9. Explain error handling and security middleware integration in a web application. Describe how authentication errors, authorization errors, validation errors, and server errors should be handled and logged.

5 Marks

Security middleware

- **helmet** – sets secure HTTP headers.
- **cors** – controls which domains can access the API.
- **express-rate-limit** – limits repeated requests (stops brute force).
- **express.json({limit})** – limits body size.
- Input validation and sanitization (express-validator).

```
const helmet = require('helmet');
```

```
const cors = require('cors');
const rateLimit = require('express-rate-limit');

app.use(helmet());
app.use(cors());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
app.use(express.json());
```

How each error type is handled

Error type	Status	Handling	Log level
Validation error	400	Return list of invalid fields	warn
Authentication error	401	Missing/wrong token or password	warn
Authorization error	403	Role not allowed	warn
Not found	404	Route/resource missing	info
Server error	500	Generic message to client; full details in log	error

Centralized error classes and handler

```
class AppError extends Error {
  constructor(message, status) { super(message); this.status = status; }
}

// usage in routes / middleware
if (!token) throw new AppError('Login required', 401);
if (user.role !== 'admin') throw new AppError('Access denied', 403);
if (!name) throw new AppError('Name is required', 400);

// final error middleware (after all routes)
app.use((err, req, res, next) => {
  const status = err.status || 500;
  console.error(`[${status}] ${req.method} ${req.url} - ${err.stack}`);

  res.status(status).json({
    success: false,
    message: status === 500 ? 'Internal Server Error' : err.message
  });
});
```

Best practices

- Do not reveal internal details (stack trace, DB errors) to the client.
- Log every error with time, route and message.
- Handle async errors with try/catch + next(err).
- Never log passwords or tokens.

Q10. Develop a complete authentication and authorization workflow using JWT and middleware. Explain user registration, login, JWT generation, protected routes, custom middleware, and error handling.

5 Marks

Setup

```
npm install express mongoose bcryptjs jsonwebtoken dotenv

# .env
JWT_SECRET=mysecretkey123
PORT=3000
```

Complete code (app.js)

```
require('dotenv').config();
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());
```

```

const users = []; // (use a real database in projects)

// ---- Custom middleware: logger ----
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

// ---- 1. REGISTER ----
app.post('/register', async (req, res, next) => {
  try {
    const { name, email, password, role } = req.body;
    if (!name || !email || !password)
      return res.status(400).json({ message: 'All fields required' });
    if (users.find(u => u.email === email))
      return res.status(409).json({ message: 'User exists' });

    const hashed = await bcrypt.hash(password, 10);
    users.push({ id: users.length + 1, name, email,
      password: hashed, role: role || 'user' });
    res.status(201).json({ message: 'Registered' });
  } catch (e) { next(e); }
});

// ---- 2. LOGIN (JWT generation) ----
app.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;
    const user = users.find(u => u.email === email);
    if (!user || !(await bcrypt.compare(password, user.password)))
      return res.status(401).json({ message: 'Invalid credentials' });

    const token = jwt.sign({ id: user.id, role: user.role },
      process.env.JWT_SECRET, { expiresIn: '1h' });
    res.json({ token });
  } catch (e) { next(e); }
});

// ---- 3. Authentication middleware ----
const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(403).json({ message: 'Invalid/expired token' }); }
};

// ---- 4. Authorization middleware (roles) ----
const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next()
  : res.status(403).json({ message: 'Access denied' });

// ---- 5. Protected routes ----
app.get('/profile', authenticate, (req, res) => res.json(req.user));
app.get('/admin', authenticate, authorize('admin'),
  (req, res) => res.json({ message: 'Welcome Admin' }));

// ---- 6. Error handling ----
app.use((req, res) => res.status(404).json({ message: 'Not found' }));
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Server error' });
});

app.listen(process.env.PORT, () => console.log('Running...'));

```

Workflow explanation

- **Registration:** validate -> check duplicate -> hash password with bcrypt -> save user.
- **Login:** find user -> bcrypt.compare -> create JWT with id and role -> send token.
- **Protected route:** client sends Authorization: Bearer <token>; authenticate verifies it.
- **Authorization:** authorize('admin') allows only admin role.
- **Custom middleware:** logger, authenticate, authorize.
- **Error handling:** 400, 401, 403, 404 handled in routes; 500 handled by the global error middleware.

MODULE-IV : File System, Streams and Buffers

PART A - SHORT QUESTIONS (2 Marks each)

Q1. What is fs module in Node.js?

2 Marks

The fs (File System) module is a **built-in Node.js module used to work with files and directories** - create, read, write, append, rename and delete.

- Imported using: `const fs = require('fs');`
- Has both **asynchronous** (callback/promise) and **synchronous** versions of methods (`readFile`, `readFileSync`).

Q2. What is the difference between fs.readFile() and fs.createReadStream()?

2 Marks

Point	fs.readFile()	fs.createReadStream()
Reading	Reads whole file into memory at once	Reads file in small chunks
Memory	High for big files	Low (efficient)
Use	Small files	Large files (videos, logs)
Result	Callback with full data	Events: data, end, error

Q3. What is a Readable Stream?

2 Marks

A Readable stream is a stream from which **data can be read in chunks**.

- Examples: `fs.createReadStream()`, HTTP request (`req`), `process.stdin`.
- Important events: **data**, **end**, **error**.

Q4. What is a Writable Stream?

2 Marks

A Writable stream is a stream to which **data can be written in chunks**.

- Examples: `fs.createWriteStream()`, HTTP response (`res`), `process.stdout`.
- Methods: **write()**, **end()**; events: **finish**, **error**.

Q5. What is a Duplex Stream?

2 Marks

A Duplex stream is **both readable and writable**; the two sides work independently.

- Example: **TCP socket** (`net.Socket`), `WebSocket`.
- Data can be read and written at the same time.

Q6. What is a Transform Stream?

2 Marks

A Transform stream is a special **Duplex stream that modifies (transforms) data while it passes through**.

- Example: `zlib.createGzip()` (compression), converting text to uppercase.
- Implemented using the `transform(chunk, encoding, callback)` method.

Q7. What is the purpose of the pipe() method?

2 Marks

`pipe()` **connects a Readable stream to a Writable stream**, so data flows automatically from source to destination.

- Example: `fs.createReadStream('a.txt').pipe(fs.createWriteStream('b.txt'));`
- It manages flow control (backpressure) and saves memory.

Q8. What is a Buffer in Node.js?**2 Marks**

A Buffer is a **temporary memory area that stores raw binary data** (bytes). It is used to handle files, network data and streams.

- Example: `const buf = Buffer.from('Hello'); console.log(buf.toString()); // Hello`
- Buffer size is fixed once created.

Q9. How is the "error" event used in stream error handling?**2 Marks**

Streams are EventEmitters. If an error occurs (e.g. file not found), the stream emits an **'error' event**. We handle it with a listener, otherwise the program crashes.

- Example: `stream.on('error', (err) => console.log(err.message));`

Q10. What is the difference between Promise-based and Callback-based asynchronous file operations?**2 Marks**

Point	Callback-based (fs)	Promise-based (fs/promises)
Import	<code>require('fs')</code>	<code>require('fs/promises')</code>
Style	Pass a callback function	Returns a Promise (then/await)
Error handling	if (err) inside callback	<code>try...catch</code> or <code>.catch()</code>
Readability	Callback hell for nested calls	Clean with <code>async/await</code>

PART B - LONG QUESTIONS (5 Marks each)

Q1. Explain the Node.js File System (fs) module and discuss the various file operations such as creating, reading, appending, renaming and deleting files with suitable examples.

5 Marks

Introduction

The **fs module** is built into Node.js and is used to interact with the file system. Import: **const fs = require('fs');** Each method has an **async (callback)** version and a **sync** version (ends with Sync). Async versions are preferred as they do not block the server.

1. Creating / writing a file - writeFile()

Creates the file if it does not exist; **overwrites** the content if it exists.

```
fs.writeFile('demo.txt', 'Hello Node.js', (err) => {
  if (err) throw err;
  console.log('File created');
});
```

2. Reading a file - readFile()

```
fs.readFile('demo.txt', 'utf8', (err, data) => {
  if (err) return console.log(err.message);
  console.log(data);          // Hello Node.js
});
```

3. Appending - appendFile()

Adds new content at the end without removing old content.

```
fs.appendFile('demo.txt', '\nWelcome to Backend', (err) => {
  if (err) throw err;
  console.log('Data appended');
});
```

4. Renaming - rename()

```
fs.rename('demo.txt', 'newdemo.txt', (err) => {
  if (err) throw err;
  console.log('File renamed');
});
```

5. Deleting - unlink()

```
fs.unlink('newdemo.txt', (err) => {
  if (err) throw err;
  console.log('File deleted');
});
```

Summary

Operation	Method
Create / Write	fs.writeFile()
Read	fs.readFile()
Append	fs.appendFile()
Rename	fs.rename()
Delete	fs.unlink()
Check exists	fs.existsSync()

Sync version example: `const data = fs.readFileSync('demo.txt', 'utf8');` (blocks the program until finished).

Q2. Explain the directory operations available in Node.js using the fs module. Discuss how to create, read and delete directories with appropriate methods and examples.

5 Marks

Directories (folders) are also handled by the fs module. Main operations:

1. Create a directory - fs.mkdir()

```
fs.mkdir('myFolder', (err) => {
  if (err) throw err;
  console.log('Directory created');
});

// nested folders: use recursive option
fs.mkdir('a/b/c', { recursive: true }, (err) => {
  if (err) throw err;
});
```

2. Read a directory - fs.readdir()

Returns an array of file and folder names present in the directory.

```
fs.readdir('myFolder', (err, files) => {
  if (err) throw err;
  console.log(files);           // [ 'a.txt', 'b.txt' ]
});
```

3. Delete a directory

```
// empty directory
fs.rmdir('myFolder', (err) => {
  if (err) throw err;
  console.log('Directory deleted');
});

// directory with files (recommended)
fs.rm('myFolder', { recursive: true, force: true }, (err) => {
  if (err) throw err;
});
```

4. Other useful methods

- **fs.existsSync(path)** - check whether a folder/file exists.
- **fs.stat(path)** - get details (size, isDirectory(), isFile()).
- **fs.rename(old, new)** - rename a directory.

Example - create folder, add file, list

```
const fs = require('fs');
if (!fs.existsSync('data')) fs.mkdirSync('data');
fs.writeFileSync('data/info.txt', 'Hello');
console.log(fs.readdirSync('data')); // [ 'info.txt' ]
```

Operation	Method
Create	mkdir()
Read	readdir()
Delete empty	rmdir()
Delete with contents	rm({recursive:true})

Q3. What are streams in Node.js? Explain their advantages and discuss the different types of streams - Readable, Writable, Duplex and Transform streams.

5 Marks

Definition

A stream is a way of **handling data continuously in small pieces (chunks)** instead of loading the whole data into memory at once. Streams are objects of **EventEmitter** and are available in the **stream** module. Example: watching a YouTube video - it plays while the data is still being downloaded.

Advantages

- **Memory efficient** - only a small chunk is in memory.
- **Faster** - processing starts as soon as the first chunk arrives.
- Handles **very large files** easily.
- **Composable** - streams can be connected using pipe().

Types of streams

- **1. Readable stream** - source of data; we read from it. Example: fs.createReadStream(), http request. Events: data, end, error.
- **2. Writable stream** - destination; we write to it. Example: fs.createWriteStream(), http response. Methods: write(), end(); event: finish.
- **3. Duplex stream** - readable + writable at the same time. Example: TCP socket.
- **4. Transform stream** - a duplex stream that changes the data while passing it. Example: zlib.createGzip(), uppercase converter.

Diagram

Readable --> Transform --> Writable
(file.txt) (uppercase) (out.txt)

Quick example

```
const fs = require('fs');
const rs = fs.createReadStream('big.txt');
const ws = fs.createWriteStream('copy.txt');
rs.pipe(ws); // copies file chunk by chunk
```

Type	Direction	Example
Readable	Read only	createReadStream
Writable	Write only	createWriteStream
Duplex	Read + Write	net.Socket
Transform	Read + Write + modify	zlib.createGzip

Q4. Explain how to work with Readable and Writable file streams in Node.js. Describe the data, end and finish events with suitable examples.

5 Marks

Readable file stream

Created using **fs.createReadStream(path, options)**. It reads the file in chunks (default 64 KB).

- **'data'** - emitted when a chunk is available.
- **'end'** - emitted when there is no more data to read.
- **'error'** - emitted if an error occurs (e.g. file not found).

```
const fs = require('fs');
const rs = fs.createReadStream('input.txt', { encoding: 'utf8' });

rs.on('data', (chunk) => {
  console.log('Received chunk of length', chunk.length);
});
rs.on('end', () => console.log('Reading finished'));
rs.on('error', (err) => console.log('Error:', err.message));
```

Writable file stream

Created using **fs.createWriteStream(path)**. Data is written using **write()** and the stream is closed using **end()**.

- **'finish'** - emitted after **end()** is called and all data is flushed to the file.
- **'error'** - emitted on write errors.

```
const ws = fs.createWriteStream('output.txt');
ws.write('Hello ');
ws.write('Node.js Streams\n');
ws.end('Done'); // no more data

ws.on('finish', () => console.log('All data written'));
ws.on('error', (err) => console.log(err.message));
```

Copy file using both

```
const rs = fs.createReadStream('input.txt');
const ws = fs.createWriteStream('copy.txt');

rs.on('data', (chunk) => ws.write(chunk)); // data event
rs.on('end', () => ws.end()); // end event
ws.on('finish', () => console.log('Copied')); // finish event
```

Event	Stream	When it occurs
data	Readable	A chunk is available
end	Readable	No more data to read
finish	Writable	All data written after end()
error	Both	Any error

Q5.

Explain Duplex Streams in Node.js. How do their readable and writable sides work independently? Illustrate with an example.

5 Marks

Definition

A Duplex stream implements **both Readable and Writable interfaces**. It can receive data (write) and give data (read) at the same time. Imported from the **stream** module as **Duplex**.

Independent sides

- **Writable side** - data written with **write()** goes into its own internal buffer; handled by the **write()** method.
- **Readable side** - data is produced using **push()** and consumed with 'data' event or **read()**; handled by the **read()** method.
- The two sides have **separate buffers**. What is written need not be what is read (unlike Transform).
- Real example: **TCP socket** - data you send (write) and data you receive (read) are different.

```
      write(data) --> [ Writable side ] (internal buffer 1)
Duplex stream
      'data' <--- [ Readable side ] (internal buffer 2)
```

Example

```
const { Duplex } = require('stream');

const myDuplex = new Duplex({
  // Readable side: produce data
  read(size) {
    this.push('Data from readable side\n');
    this.push(null); // end of reading
  },
  // Writable side: consume data
  write(chunk, encoding, callback) {
    console.log('Writable got:', chunk.toString());
    callback();
  }
});

myDuplex.on('data', (chunk) => console.log('Read:', chunk.toString()));
myDuplex.write('Hello Duplex');
myDuplex.end();
```

Output

Writable got: Hello Duplex
Read: Data from readable side

This shows that writing 'Hello Duplex' and reading 'Data from readable side' are two independent operations.

Q6. What are Data Transformation Streams? Explain how a Transform Stream modifies data while it is being transferred. Develop an example to convert file content to uppercase.

5 Marks

Definition

A Transform stream is a **Duplex stream whose output is computed from its input**. Data written into it is **modified and then made available for reading**. It sits between a Readable and a Writable stream.

```
input.txt --> Readable --> Transform (UPPERCASE) --> Writable --> output.txt
```

How it works

- Create it using **new Transform({ transform(chunk, encoding, callback) {...} })**.
- Each incoming **chunk** is converted (e.g. toUpperCase()).
- The result is passed forward using **callback(null, newData)** (or this.push()).
- Then connected using pipe().

Example - convert file content to uppercase

```
const fs = require('fs');
const { Transform } = require('stream');

// Transform stream
const toUpper = new Transform({
  transform(chunk, encoding, callback) {
    const upper = chunk.toString().toUpperCase();
    callback(null, upper); // send modified data
  }
});

const readStream = fs.createReadStream('input.txt');
const writeStream = fs.createWriteStream('output.txt');

readStream.pipe(toUpper).pipe(writeStream);

writeStream.on('finish', () => console.log('File converted to uppercase'));
```

Result

input.txt	output.txt
hello node js	HELLO NODE JS

Other uses

Compression (zlib.createGzip), encryption (crypto), parsing CSV/JSON, data cleaning.

Q7. Explain error handling in Node.js streams. Discuss the use of .on('error') and pipeline() with suitable examples.

5 Marks

Why needed

Streams may fail (file not found, permission denied, network break). If an **'error' event is not handled, the Node.js process crashes**.

1. Using .on('error')

Attach an error listener to **every stream**.

```
const fs = require('fs');
const rs = fs.createReadStream('missing.txt');
const ws = fs.createWriteStream('out.txt');

rs.on('error', (err) => console.log('Read error:', err.message));
```

```
ws.on('error', (err) => console.log('Write error:', err.message));

rs.pipe(ws);
```

Problem with pipe(): errors are NOT passed along the chain, and if one stream fails the others are not closed automatically (may cause memory leaks).

2. Using pipeline() (recommended)

stream.pipeline() connects streams, **forwards errors to a single callback and destroys all streams properly** on failure.

```
const { pipeline } = require('stream');
const zlib = require('zlib');

pipeline(
  fs.createReadStream('input.txt'),
  zlib.createGzip(),
  fs.createWriteStream('input.txt.gz'),
  (err) => {
    if (err) console.log('Pipeline failed:', err.message);
    else console.log('Pipeline succeeded');
  }
);
```

3. Promise version

```
const { pipeline } = require('stream/promises');
try {
  await pipeline(readStream, transformStream, writeStream);
} catch (err) {
  console.log('Error:', err.message);
}
```

Point	pipe() + on('error')	pipeline()
Error handling	Separate for each stream	One callback
Cleanup	Manual	Automatic
Recommended	Simple cases	Production code

Q8. Explain Stream Piping in Node.js. Discuss how data is transferred from a Readable Stream to a Writable Stream and explain its advantages with examples.

5 Marks

Definition

Piping is the mechanism of **connecting the output of a Readable stream to the input of a Writable stream** using the **pipe()** method. Syntax: **readable.pipe(writable)**. It returns the destination stream, so pipes can be **chained**.

How data is transferred

- Readable stream reads a chunk from the source (file).
- pipe() automatically passes that chunk to the writable stream's write().
- If the writable is slow, pipe() **pauses** the readable (**backpressure**) and resumes when the writable is ready.
- When the source ends, pipe() calls end() on the writable automatically.

Source file --> [Readable] --chunks--> pipe() --> [Writable] --> Destination file

Example 1 - copy a file

```
const fs = require('fs');
fs.createReadStream('source.txt').pipe(fs.createWriteStream('dest.txt'));
```


Example 2 - chaining (compress a file)

```
const zlib = require('zlib');
fs.createReadStream('data.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('data.txt.gz'));
```

Example 3 - send a big file in an HTTP response

```
const http = require('http');
http.createServer((req, res) => {
  fs.createReadStream('video.mp4').pipe(res); // streams to client
}).listen(3000);
```

Advantages

- Very **little code**.
- **Memory efficient** - works for huge files.
- **Backpressure** handled automatically.
- Easy to chain multiple operations (read -> compress -> write).

Q9. Explain Buffer Management in Node.js. Discuss Buffer creation, reading, writing, converting Buffers to strings and the role of Buffers in Streams.

5 Marks

Definition

A **Buffer** is a **fixed-size temporary memory area that stores raw binary data** (a sequence of bytes). It is a global class in Node.js (no import needed) and is used for files, network packets and streams. Each element is a byte (0 - 255).

1. Creating buffers

```
const b1 = Buffer.alloc(5); // 5 bytes filled with 0
const b2 = Buffer.from('Hello'); // from a string
const b3 = Buffer.from([72, 101, 108]); // from an array of bytes
console.log(b2); // <Buffer 48 65 6c 6c 6f>
```

2. Writing to a buffer

```
const buf = Buffer.alloc(10);
buf.write('Node'); // writes text
buf[4] = 33; // write one byte (33 = '!')
console.log(buf.toString()); // Node!
```

3. Reading from a buffer

```
const b = Buffer.from('Hello');
console.log(b[0]); // 72 (byte at index 0)
console.log(b.length); // 5
console.log(b.slice(0, 2)); // <Buffer 48 65>
```

4. Converting buffer to string (and back)

```
const b = Buffer.from('Hello Node');
console.log(b.toString()); // Hello Node (utf8)
console.log(b.toString('hex')); // 48656c6c6f204e6f6465
console.log(b.toString('base64')); // SGVsbG8gTm9kZQ==
console.log(b.toJSON()); // { type: 'Buffer', data: [...] }
```

5. Other methods

Buffer.concat([b1, b2]) joins buffers; Buffer.byteLength(str) gives size; buf.equals(other) compares.

Role of Buffers in Streams

- Streams move data as **chunks, and each chunk is a Buffer** (unless an encoding such as 'utf8' is set).
- Buffer holds the chunk **temporarily** until the next step (write to file / send over network) is ready.
- Streams have an internal buffer limit (**highWaterMark**, default 16 KB for streams, 64 KB for fs read); when it is full, backpressure pauses the source.

```
rs.on('data', (chunk) => {
```

```
console.log(Buffer.isBuffer(chunk)); // true
console.log(chunk.toString());
});
```

Q10.

Explain Asynchronous File Operations in Node.js. Compare Promise-based (fs/promises) and Callback-based (fs) approaches with suitable examples. Discuss the advantages of using async/await and the problem of callback hell.

5 Marks

Asynchronous file operations

Async operations **do not block** the program. Node.js starts the file operation, continues executing other code, and gets notified when the operation completes. This keeps the server responsive for other users.

1. Callback-based (fs)

```
const fs = require('fs');
fs.readFile('a.txt', 'utf8', (err, data) => {
  if (err) return console.log(err.message);
  console.log(data);
});
console.log('This prints first');
```

2. Promise-based (fs/promises) with async/await

```
const fs = require('fs/promises');

async function readFileData() {
  try {
    const data = await fs.readFile('a.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.log('Error:', err.message);
  }
}
readFileData();
```

Comparison

Point	Callback (fs)	Promise (fs/promises)
Import	require('fs')	require('fs/promises')
Result	Via callback (err, data)	Returns a Promise
Errors	if (err) in each callback	try...catch / .catch()
Chaining	Nested callbacks	then() or await in sequence
Readability	Poor when nested	Good (looks synchronous)

Callback hell

When many async operations depend on each other, callbacks get nested deeply (**pyramid of doom**), making code hard to read, debug and maintain.

```
fs.readFile('a.txt', 'utf8', (e1, a) => {
  fs.readFile('b.txt', 'utf8', (e2, b) => {
    fs.writeFile('c.txt', a + b, (e3) => {
      fs.appendFile('log.txt', 'done', (e4) => {
        console.log('Finished'); // deeply nested
      });
    });
  });
});
```

Same code with async/await

```
async function merge() {
  try {
    const a = await fs.readFile('a.txt', 'utf8');
    const b = await fs.readFile('b.txt', 'utf8');
    await fs.writeFile('c.txt', a + b);
    await fs.appendFile('log.txt', 'done');
    console.log('Finished');
  }
}
```

```
    } catch (err) { console.log(err.message); }  
  }  
  merge();
```

Advantages of async/await

- Clean, flat and readable code.
 - Single try...catch for all errors.
 - Easy to debug and maintain.
 - Avoids callback hell.
 - Promise.all() can run operations in parallel.
-

MODULE-V : MongoDB, Mongoose, MVC and JWT Authentication

PART A - SHORT QUESTIONS (2 Marks each)

Q1. What is MongoDB? Mention any two advantages of MongoDB.

2 Marks

MongoDB is an **open-source NoSQL, document-oriented database** that stores data as **JSON-like documents (BSON)** in collections instead of tables and rows.

Advantages:

- **Schema-flexible** – documents in a collection can have different fields.
- **Scalable and fast** – supports horizontal scaling (sharding) and easy to use with JavaScript/Node.js.

Q2. What is MongoDB Atlas? Why is it used?

2 Marks

MongoDB Atlas is the **fully managed cloud database service** of MongoDB (runs on AWS, Azure, Google Cloud).

Used because:

- No need to install or maintain MongoDB on our own computer; it gives a connection string.
- Provides automatic backup, security, scaling and a free tier for learning.

Q3. What is the use of the .env file in a Node.js application?

2 Marks

The .env file **stores environment variables / secret configuration** such as database URL, JWT secret and port number as KEY=VALUE pairs.

- Loaded using the **dotenv** package: `require('dotenv').config();` and accessed by `process.env.KEY`.
- Keeps secrets out of the source code; the file is added to `.gitignore`.

Q4. What is Mongoose? Mention one advantage of using Mongoose.

2 Marks

Mongoose is an **ODM (Object Data Modeling) library for MongoDB and Node.js**. It lets us define schemas and models and work with MongoDB using JavaScript objects.

- Advantage: provides **schema definition and built-in validation** (required, min, unique), so data stays consistent.

Q5. What is a Schema in Mongoose? Why is it used?

2 Marks

A Schema defines the **structure of documents** in a collection – field names, data types, default values and validation rules.

- Example: `new mongoose.Schema({ name: String, age: { type: Number, min: 18 } });`
- Used to keep data consistent and validated before it is saved.

Q6. What is the difference between find() and findById() in Mongoose?

2 Marks

Point	find()	findById()
Purpose	Gets all documents matching a condition	Gets one document using its <code>_id</code>
Returns	Array of documents	Single document (or null)
Example	<code>User.find({ age: 20 })</code>	<code>User.findById(id)</code>

Q7. What is the difference between a Mongoose Schema and a Mongoose Model?**2 Marks**

Point	Schema	Model
Meaning	Blueprint / structure of the document	Constructor created from the schema
Role	Defines fields, types, validation	Used to perform database operations (CRUD)
Created by	<code>new mongoose.Schema({...})</code>	<code>mongoose.model('User', schema)</code>

Q8. What is password hashing? Why should passwords not be stored in plain text?**2 Marks**

Password hashing is converting a password into a **fixed-length, irreversible string (hash)** using an algorithm such as bcrypt (with salt).

Not stored in plain text because:

- If the database is hacked, attackers can read all passwords directly.
- Hashes cannot be reversed, so user passwords stay safe.

Q9. What is JWT? Why is it used in authentication?**2 Marks**

JWT (JSON Web Token) is a **signed token made of header, payload and signature** that proves the identity of a user.

- Server creates it after login; the client sends it with each request.
- Used because it is **stateless, secure (signed), compact** and no session storage is required.

Q10. What is the difference between authentication and authorization?**2 Marks**

Authentication	Authorization
Verifies WHO the user is	Decides WHAT the user can access
Done first (login)	Done after authentication
Example: email + password	Example: only admin can delete users

PART B - LONG QUESTIONS (5 Marks each)

Q1. Explain the steps to connect MongoDB Atlas with a Node.js Express application.

5 Marks

Steps

- **Step 1 - Create account:** Sign up on mongodb.com/atlas.
- **Step 2 - Create a cluster:** choose the free (M0) tier, cloud provider and region.
- **Step 3 - Create database user:** Database Access -> Add user (username + password).
- **Step 4 - Network access:** Network Access -> Add IP address (your IP or 0.0.0.0/0 for testing).
- **Step 5 - Get connection string:** Cluster -> Connect -> Drivers -> copy the URI.
- **Step 6 - Install packages:** `npm install express mongoose dotenv`
- **Step 7 - Store the URI in .env** and connect using `mongoose.connect()`.

.env file

```
MONGO_URI=mongodb+srv://myuser:mypassword@cluster0.abcde.mongodb.net/mydb
PORT=3000
```

Connection code (app.js)

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log('MongoDB Atlas connected');
    app.listen(process.env.PORT, () => console.log('Server running'));
  })
  .catch((err) => console.log('Connection error:', err.message));
```

Points to remember

- Add .env to .gitignore - never upload passwords to GitHub.
- If connection fails, check username/password, IP whitelist and internet.
- The database name (mydb) is given at the end of the URI.

Q2. What is Mongoose? Explain any four advantages of using Mongoose with MongoDB.

5 Marks

Definition

Mongoose is an **Object Data Modeling (ODM) library** for MongoDB and Node.js. It provides a structured way to define data using **schemas and models** and gives easy methods (create, find, update, delete) to talk to MongoDB. Install: `npm install mongoose`.

Advantages (any four)

- **1. Schema definition:** MongoDB is schema-less; Mongoose lets us define the structure (fields, types, defaults) so data is organized.
- **2. Built-in validation:** rules like required, min, max, unique, enum, match are checked automatically before saving.
- **3. Easy CRUD methods:** `create()`, `find()`, `findById()`, `findByIdAndUpdate()`, `findByIdAndDelete()` make code short and simple.
- **4. Middleware (hooks):** `pre('save')` and `post('save')` functions run automatically, e.g. to hash passwords before saving.
- **5. Relationships (populate):** can link documents of different collections.
- **6. Query building and promises/async-await support.**

Example

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 18 }
});
const User = mongoose.model('User', userSchema);

await User.create({ name: 'Ravi', email: 'ravi@mail.com', age: 21 });
```

Q3.

Explain the MVC architecture and describe the responsibilities of the Model, View and Controller with a suitable example.

5 Marks

Definition

MVC (**Model-View-Controller**) is a design pattern that divides an application into **three parts** so that code is organized, reusable and easy to maintain.

Components

- **Model** - manages **data and database logic** (Mongoose schema and model). Example: User model.
- **View** - the **user interface** shown to the user (HTML, EJS, React, or JSON response in APIs).
- **Controller** - the **middle layer**: receives the request from the route, uses the Model to get/save data and sends the response (View).

Flow diagram

```
Client --request--> Route --> Controller --> Model --> Database
Client <--response-- View <-- Controller <-- Model <-- Database
```

Example folder structure

```
project/
|-- models/userModel.js
|-- controllers/userController.js
|-- routes/userRoutes.js
|-- views/ (or frontend)
|-- app.js
```

Example code

```
// models/userModel.js (MODEL)
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({ name: String, email: String });
module.exports = mongoose.model('User', userSchema);

// controllers/userController.js (CONTROLLER)
const User = require('../models/userModel');
exports.getUsers = async (req, res) => {
  const users = await User.find();
  res.json(users); // VIEW (JSON response)
};

// routes/userRoutes.js
const router = require('express').Router();
router.get('/users', require('../controllers/userController').getUsers);
module.exports = router;

// app.js
app.use('/api', require('./routes/userRoutes'));
```

Advantages

- Separation of concerns.
- Easy maintenance and testing.
- Teamwork is easier (different people work on M, V, C).
- Code reusability.

Steps

- **1.** Install and import mongoose.
- **2.** Create a schema with **new mongoose.Schema({...})** defining fields, types and validations.
- **3.** Create a model with **mongoose.model('User', userSchema)**.
- **4.** Export the model and use it in controllers.

models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  role: {
    type: String,
    enum: ['user', 'admin'],
    default: 'user'
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

const User = mongoose.model('User', userSchema);
module.exports = User;
```

Explanation

- **type** – data type (String, Number, Date, Boolean).
- **required** – field must be present.
- **unique** – no duplicate values (creates an index).
- **default** – value used when none is given.
- **enum** – only allowed values.
- The model name 'User' automatically maps to the **'users'** collection in MongoDB.

Using the model

```
const User = require('./models/User');
const u = new User({ name: 'Ravi', email: 'ravi@mail.com', password: '123456' });
await u.save();
```


Q5.**Explain how to perform Create, Read, Update and Delete operations on users using Mongoose.****5 Marks**

Assume the User model (name, email, age) is already created and **app.use(express.json())** is used.

1. Create

```
app.post('/users', async (req, res) => {
  try {
    const user = await User.create(req.body); // or new User(req.body).save()
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
```

2. Read

```
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.status(200).json(users);
});

app.get('/users/:id', async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});
```

3. Update

```
app.put('/users/:id', async (req, res) => {
  const user = await User.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true, runValidators: true } // return updated doc
  );
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});
```

4. Delete

```
app.delete('/users/:id', async (req, res) => {
  const user = await User.findByIdAndDelete(req.params.id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json({ message: 'User deleted' });
});
```

CRUD	Mongoose method	HTTP	Status
Create	create() / save()	POST	201
Read	find() / findById()	GET	200
Update	findByIdAndUpdate()	PUT	200
Delete	findByIdAndDelete()	DELETE	200

Q6.**Explain how to get all users and get a single user by ID using Mongoose.****5 Marks**

1. Get all users - find()

User.find() returns an **array of all documents**. If nothing is found, it returns an empty array [].

```
app.get('/users', async (req, res) => {
  try {
    const users = await User.find(); // all users
    // const users = await User.find().select('-password'); // hide password
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

2. Get single user by ID - findById()

The id is taken from the **route parameter** (req.params.id). findById() returns **one document or null**.

```
app.get('/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.status(200).json(user);
  } catch (err) {
    res.status(400).json({ message: 'Invalid user id' });
  }
});
```

Outputs

- GET /users -> [{..user1..}, {..user2..}]
- GET /users/652ab1c9... -> { _id, name, email }
- GET /users/00000000000000000000000000000000 -> 404 User not found
- GET /users/abc -> 400 Invalid user id (CastError)

Other: User.findOne({ email }) returns the first user matching a condition.

Q7.

Explain how to use query parameters in Mongoose to filter products based on price and category, and search products by name.

5 Marks

Product schema

```
const productSchema = new mongoose.Schema({
  name: String, category: String, price: Number
});
const Product = mongoose.model('Product', productSchema);
```

Idea

Read the values from **req.query**, build a **filter object** only with the parameters that are present, and pass it to **Product.find(filter)**.

Code

```
app.get('/products', async (req, res) => {
  try {
    const { category, minPrice, maxPrice, search } = req.query;
    const filter = {};

    // filter by category
    if (category) filter.category = category;

    // filter by price range
    if (minPrice || maxPrice) {
      filter.price = {};
      if (minPrice) filter.price.$gte = Number(minPrice);
      if (maxPrice) filter.price.$lte = Number(maxPrice);
    }

    // search by name (case-insensitive, partial match)
    if (search) filter.name = { $regex: search, $options: 'i' };

    const products = await Product.find(filter);
    res.json(products);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Sample requests

Request URL	Result
/products?category=mobile	All mobile products
/products?minPrice=10000&maxPrice=30000	Price between 10000 and 30000

Request URL	Result
/products?search=sam	Names containing 'sam' (Samsung...)
/products?category=mobile&maxPrice=20000&search=re	Combination of all

\$gte = greater than or equal, **\$lte** = less than or equal, **\$regex** = pattern search, **\$options:'i'** = ignore case.

Q8. Explain the purpose of HTTP status codes 200, 201, 400, 401, 403, 404 and 500 in REST APIs.

5 Marks

HTTP status codes are **3-digit numbers sent by the server in the response** to tell the client the result of the request. They help the client understand whether the request succeeded or failed.

Code	Name	Purpose	Example
200	OK	Request succeeded	GET /users returns data
201	Created	New resource created successfully	POST /users - user registered
400	Bad Request	Invalid or missing data from client	Email missing in body
401	Unauthorized	Not authenticated (no/invalid token or credentials)	Wrong password; no JWT
403	Forbidden	Authenticated but not permitted	Normal user opens admin route
404	Not Found	Resource / URL does not exist	GET /users/999 not found
500	Internal Server Error	Unexpected error on server	DB connection failure, bug

Categories

- **2xx** - Success
- **3xx** - Redirection
- **4xx** - Client errors (200 series is success; 400-499 mistakes by client)
- **5xx** - Server errors

Example in Express

```
res.status(200).json(users); // OK
res.status(201).json({ message: 'User created' }); // Created
res.status(400).json({ error: 'Email is required' }); // Bad request
res.status(401).json({ error: 'Invalid credentials' }); // Unauthorized
res.status(403).json({ error: 'Access denied' }); // Forbidden
res.status(404).json({ error: 'User not found' }); // Not found
res.status(500).json({ error: 'Server error' }); // Server error
```

Tip to remember: 401 = 'Who are you?' (login problem); 403 = 'I know you, but you cannot enter' (permission problem).

Q9. Explain the process of user registration and login using JWT authentication. Describe the major steps from registration to generating a JWT token after successful login.

5 Marks

Setup

```
npm install express mongoose bcryptjs jsonwebtoken dotenv
# .env -> JWT_SECRET=mysecretkey
```

Part A - Registration (POST /api/register)

- 1. Client sends name, email, password.
- 2. Server validates the input (400 if missing).
- 3. Server checks whether the email already exists (409/400).
- 4. Password is **hashed** using bcryptjs.
- 5. User is saved in MongoDB with the hashed password.
- 6. Server sends 201 Created.

```
exports.register = async (req, res) => {
  try {
    const { name, email, password } = req.body;
    if (!name || !email || !password)
      return res.status(400).json({ message: 'All fields are required' });

    const exist = await User.findOne({ email });
    if (exist) return res.status(400).json({ message: 'User already exists' });

    const hashed = await bcrypt.hash(password, 10);
    await User.create({ name, email, password: hashed });
    res.status(201).json({ message: 'User registered successfully' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

Part B - Login (POST /api/login)

- 1. Client sends email and password.
- 2. Server finds user by email (401 if not found).
- 3. bcrypt.compare(password, user.password) checks the password (401 if wrong).
- 4. If correct, server creates a **JWT** using jwt.sign() with payload {id, role}, secret and expiry.
- 5. Token is sent to the client (200 OK).
- 6. Client sends it in later requests: Authorization: Bearer <token>.

```
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user)
      return res.status(401).json({ message: 'Invalid email or password' });

    const match = await bcrypt.compare(password, user.password);
    if (!match)
      return res.status(401).json({ message: 'Invalid email or password' });

    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '1h' }
    );
    res.status(200).json({ message: 'Login successful', token });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

Flow summary

Register: input -> validate -> check duplicate -> hash -> save -> 201
Login : input -> find user -> compare hash -> sign JWT -> send token

Q10. Explain why passwords should not be stored as plain text. Describe how bcryptjs can be used to hash a password during registration and compare it during login.

5 Marks

Why not plain text

- If the database is hacked or leaked, **all passwords are exposed** directly.
- Developers / admins with database access can see the passwords.
- Many people reuse the same password, so other accounts are also at risk.
- It violates security standards and data-protection laws.
- Hashing is **one-way** – the original password cannot be recovered from the hash.

What is bcryptjs

bcryptjs is a library for **hashing passwords with a salt**. **Salt** is a random string added to the password before hashing, so the same password gives a different hash each time, and it protects against rainbow-table attacks. **Salt rounds** (e.g. 10) control how slow/strong the hashing is. Install: npm install bcryptjs.

Hashing during registration - bcrypt.hash()

```
const bcrypt = require('bcryptjs');

app.post('/register', async (req, res) => {
  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10); // 10 salt rounds

  await User.create({ name, email, password: hashedPassword });
  res.status(201).json({ message: 'Registered' });
});

// stored in DB: $2a$10$N9qo8uL0ickgx2ZMRZoMy.Mrq... (not '123456')
```

Comparing during login - bcrypt.compare()

```
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(401).json({ message: 'Invalid credentials' });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

  res.json({ message: 'Login successful' });
});
```

How compare works

The hash stores the salt inside it. bcrypt.compare() takes the entered password, hashes it with the same salt and checks whether the result equals the stored hash. We **never decrypt** the hash.

Step	Function	When
Hash password	bcrypt.hash(password, 10)	Registration
Verify password	bcrypt.compare(plain, hash)	Login